

Full-Stack Prueba técnica: Multi-Tenant Micro-Architecture

1. Objetivo del Desafío

El propósito de esta prueba es evaluar tu capacidad para iniciar, estructurar y asegurar un proyecto desde cero bajo una arquitectura **Multi-Tenant (Multinquilino)**.

No vas a desarrollar un sistema completo. Construirás un **"Slice Vertical" (un caso de uso mínimo)** enfocado exclusivamente en demostrar el aislamiento de datos, control de acceso, gestión de estado global, interceptores y middlewares. El objetivo es evaluar las bases arquitectónicas que aplicarás en el día a día dentro de la empresa.

Cabe aclarar que los puntos que desconozca los puedes realizar de la manera como te quede más fácil o tu conocimiento te lo permita.

2. Requerimientos Técnicos (Stack)

- **Backend:** Node.js (Express) en JavaScript o TypeScript.
- **Base de Datos:** MySQL o Postgres (Driver nativo mysql2 o el ORM/Query Builder de tu elección).
- **Frontend:** React (Vite), react-router-dom (v6+), Redux Toolkit.
- **Tiempo estimado de ejecución:** 3 a 4 horas.

3. Entregables y Requerimientos del Código

PARTE 1: BACKEND (Node.js & MySQL)

Debes inicializar un servidor Express desde cero y configurar una base de datos MySQL local con dos tablas esenciales: tenants y records (cada registro de negocio debe estar asociado de forma obligatoria a un tenant_id).

Se requiere el desarrollo de los siguientes puntos:

- **1. Middleware de Identificación de Tenant (identifyTenant):** Un middleware de Express que intercepte la petición entrante y extraiga el contexto del Tenant actual. Para efectos de esta prueba local, debes capturar el Tenant desde una cabecera HTTP personalizada llamada X-Tenant-ID. El middleware debe validar si el Tenant existe en la base de datos y adjuntar el objeto de configuración del inquilino al ciclo de la petición (req.tenant).
- **2. Middleware de Autenticación y Roles (checkAuth):** Un middleware que simule o valide un Token JWT. Debe verificar que el usuario esté autenticado y que pertenezca legalmente al Tenant asignado en la petición, controlando el acceso por roles (ADMIN / USER).
- **3. Rutas Seguras:** Crear los siguientes endpoints bajo /api/v1/:

- GET /records -> Debe retornar **únicamente** los registros que pertenezcan al Tenant identificado en el middleware (implementar paginación básica con limit y offset).
- POST /records -> Debe insertar un nuevo registro asegurando su vinculación automática con el Tenant actual.

PARTE 2: FRONTEND (React & Estado Global)

Debes inicializar una Single Page Application (SPA) utilizando Vite y configurar la estructura de componentes, red y estado global.

- **1. Enrutamiento Dinámico (react-router-dom):** Configura las rutas de la aplicación de modo que el identificador del tenant forme parte de la URL (ejemplo: `http://localhost:3000/tenantSlug/dashboard`). Si no hay un slug válido en la URL o el usuario no está autenticado, el sistema debe proteger la vista mediante un *Layout o Route Guard* y redirigir a `/login`.
 - **2. Interceptores de Axios:** Configura una instancia personalizada de Axios. Debes crear un **Request Interceptor** que inyecte automáticamente la cabecera X-Tenant-ID en cada petición HTTP hacia el backend, obteniendo dicho ID directamente desde el estado global de la aplicación.
 - **3. Gestión de Estado Global (Redux Toolkit):**
 - Crea un `tenantSlice` para almacenar la información del inquilino activo en la sesión.
 - **Mecanismo de Seguridad (Reset):** Implementa un reducer o acción global que limpie por completo el estado de Redux en memoria cuando el usuario cambie de entorno o cierre sesión, garantizando que no queden datos residuales del tenant anterior (*Data Bleeding*).
 - **4. Optimización de Renderizado:** Crea una vista con una tabla que liste los registros (records) obtenidos desde tu API. Añade un campo de texto (Input) para filtrar localmente los elementos por nombre. Deberás aplicar técnicas de optimización (como *Debounce*, *useMemo* o *React.memo*) para demostrar cómo evitarías que la UI se vuelva lenta al manejar grandes volúmenes de datos en la aplicación real.
- ## 4. Criterios de Evaluación
- **Seguridad y Aislamiento Dinámico:** La total imposibilidad de que modificando variables en el cliente se puedan comprometer o visualizar datos de un Tenant ajeno. El backend debe gobernar la seguridad.

- **Estructura de Proyecto Limpia:** Organización de carpetas estructurada lógicamente (controladores, middlewares, servicios, componentes, slices).
- **Filosofía DRY (Don't Repeat Yourself):** Uso correcto de abstracciones (interceptores y middlewares) para centralizar la lógica repetitiva de red y seguridad.

5. Instrucciones de Entrega

1. Sube tu código a un repositorio privado (GitHub / GitLab).
2. Incluye un archivo README.md con las instrucciones paso a paso para levantar el proyecto desde cero de manera local, junto con un script .sql básico o archivo de migración para inicializar la base de datos con datos de prueba (*data dummy*).